

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Ejercicios Prácticos Integrados

Cierre y Próximos Pasos

Introducción y Repaso

- **Semana 6** nos introducimos en:
 - NumPy avanzado (**reshape**, **transpose**, **np.linalg**, etc.).
 - Primeras visualizaciones con Matplotlib (gráficos 2D y un vistazo a 3D).
- **Semana 7, Sesión 1 (Sesión 13)** cubrimos:
 - Gráficas más avanzadas (subplots múltiples, histogramas, 3D).
 - Breve introducción a **pandas** para manejar datos tabulares.
- **Objetivo de hoy:** Resolver un problema evaluado (**Problema a Evaluar**), reforzar lo aprendido y seguir practicando la visualización y manejo de arreglos.

- **Aplicar** los conceptos de **NumPy** (manipulación avanzada, aleatoriedad, álgebra lineal) y **Matplotlib** (gráficas) en un ejercicio integrador.
- **Realizar** una **evaluación parcial** en grupos, donde se resolverá un problema práctico y se subirá a **CANVAS**.
- **Discutir** resultados y dificultades tras la entrega.

Ejercicios Prácticos Integrados

Ejercicios Prácticos Integrados $\ni$ Organización de Equipos de Trabajo

- Formar **grupos de 2-3 estudiantes**
- Seleccionar ejercicios específicos (según tiempo disponible)
- Editar un **notebook compartido** en Google Colab
- **Estrategia recomendada:**
 - Discutir enfoque antes de programar
 - Anotar dudas y resolverlas en equipo
 - Probar con diferentes valores/escenarios

Meta de la Sesión

Aplicar NumPy y Matplotlib de forma integrada en problemas prácticos reales.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 1:

Primer Gráfico con NumPy



Enunciado

- Crear un array de 20 puntos entre 0 y 10 usando `np.linspace`.
- Calcular $y = x^2$ para cada punto.
- Graficar con títulos y etiquetas apropiadas.
- Agregar una grilla para mejor visualización.

Conceptos: Creación de arrays espaciados uniformemente y operaciones vectorizadas.

Aplicación: Visualización de relaciones cuadráticas en física (energía cinética, áreas, etc.).



Enunciado

- Generar 100 datos aleatorios con distribución normal (media=50, desviación=10).
- Calcular media, mediana y desviación estándar de los datos.
- Crear un histograma mostrando la distribución.
- Agregar una línea vertical indicando la media calculada.

Conceptos: Generación de datos aleatorios, estadísticas descriptivas, histogramas.

Física relevante: Análisis de datos experimentales y distribuciones de mediciones.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 3: Múltiples Funciones en Subplots



Enunciado

- Crear un array x de 0 a 2π con 100 puntos.
- Calcular $\sin(x)$ y $\cos(x)$ para cada punto.
- Graficar ambas funciones en subplots separados (2 filas, 1 columna).
- Agregar títulos, etiquetas, grillas y leyendas apropiadas.

Conceptos: Funciones trigonométricas, subplots múltiples, personalización de gráficas.

Física relevante: Ondas, oscilaciones armónicas, movimiento circular.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 4:

Análisis de Movimiento Parabólico



Enunciado

- Simular un proyectil con $v_0 = 30 \text{ m/s}$, $\theta = 45$, $g = 9.81 \text{ m/s}^2$.
- Calcular trayectoria $x(t)$ e $y(t)$ hasta el impacto.
- Graficar altura vs tiempo y trayectoria 2D en subplots separados.
- Agregar títulos, etiquetas y grillas apropiadas.

Conceptos: Funciones trigonométricas, cinemática, arrays temporales.

Física relevante: Movimiento parabólico, balística, análisis de trayectorias.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 5:

Manipulación de Matrices



Enunciado

- Crear una matriz 4×4 con números del 1 al 16 usando **reshape**.
- Extraer la diagonal principal y calcular su suma.
- Visualizar la matriz original usando **imshow** con **colormap**.
- Crear un gráfico de barras de los elementos de la diagonal.

Conceptos: Reshape, extracción de diagonal, visualización matricial.

Aplicación: Análisis de matrices de datos, tensores, mapas de calor.

Ejercicios Prácticos Integrados $\ni$ Ejercicio 6:

Comparación de Distribuciones



Enunciado

- Generar 3 conjuntos de 1000 datos: $\text{normal}(0,1)$, $\text{normal}(2,0.5)$, $\text{uniforme}(0,4)$.
- Crear histogramas superpuestos con transparencia.
- Agregar líneas verticales indicando las medias de cada distribución.
- Incluir leyenda y etiquetas descriptivas.

Conceptos: Distribuciones estadísticas, histogramas superpuestos, transparencia.

Física relevante: Análisis estadístico de mediciones, distribuciones experimentales.

Ejercicios Prácticos Integrados $\ni$ Solución 1 de Referencia:

Primer Gráfico con NumPy



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Crear array de 20 puntos entre 0 y 10
5 x = np.linspace(0, 10, 20)
6 y = x**2
7
8 # Crear la gráfica
9 plt.plot(x, y, 'bo-', markersize=6)
10 plt.xlabel('x')
11 plt.ylabel('x^2')
12 plt.title('Función Cuadrática')
13 plt.grid(True, alpha=0.3)
14 plt.show()
15
16 # Resultado: Gráfica clara de la parábola  $y = x^2$ 
17 print(f"Rango de x: [{np.min(x):.1f}, {np.max(x):.1f}]")
18 print(f"Rango de y: [{np.min(y):.1f}, {np.max(y):.1f}]")
```

Discusión: NumPy permite crear espaciados uniformes y aplicar operaciones a todo el array simultáneamente.

Ejercicios Prácticos Integrados $\ni$ Solución 2 de Referencia:

Estadísticas Básicas y Visualización



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generar datos aleatorios normales
5 np.random.seed(42) # Para reproducibilidad
6 datos = np.random.normal(50, 10, 100)
7
8 # Calcular estadísticas
9 media = np.mean(datos)
10 mediana = np.median(datos)
11 std = np.std(datos)
12
13 print(f"Media: {media:.2f}")
14 print(f"Mediana: {mediana:.2f}")
15 print(f"Desviación estándar: {std:.2f}")
```

```
16 # Crear histograma
17 plt.figure(figsize=(8, 6))
18 plt.hist(datos, bins=15, alpha=0.7,
19         color='skyblue', edgecolor='black')
20 plt.axvline(media, color='red',
21             linestyle='--', linewidth=2,
22             label=f'Media: {media:.1f}')
23 plt.xlabel('Valor')
24 plt.ylabel('Frecuencia')
25 plt.title('Distribución de Datos Aleatorios')
26 plt.legend()
27 plt.grid(True, alpha=0.3)
28 plt.show()
```

Discusión: La línea vertical permite comparar visualmente la media teórica con la distribución observada.

Ejercicios Prácticos Integrados $\ni$ Solución 3 de Referencia:

Múltiples Funciones en Subplots

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Crear array de x y calcular funciones
5 x = np.linspace(0, 2*np.pi, 100)
6 y1 = np.sin(x)
7 y2 = np.cos(x)
8
9 # Crear subplots
10 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8,
    ↪ 6))
11
12 # Subplot superior: seno
13 ax1.plot(x, y1, 'r-', linewidth=2, label='sen(x)')
14 ax1.set_title('Función Seno')
15 ax1.set_ylabel('sen(x)')
16 ax1.legend()
17 ax1.grid(True, alpha=0.3)
18
19 # Subplot inferior: coseno
20 ax2.plot(x, y2, 'b-', linewidth=2, label='cos(x)')
21 ax2.set_title('Función Coseno')
22 ax2.set_xlabel('x (radianes)')
23 ax2.set_ylabel('cos(x)')
24 ax2.legend()
25 ax2.grid(True, alpha=0.3)
26
27 # Ajustar espaciado
28 plt.tight_layout()
29 plt.show()
```

Discusión: Los subplots permiten comparar funciones relacionadas manteniendo escalas independientes.

Ejercicios Prácticos Integrados $\ni$ Solución 4 de Referencia:

Análisis de Movimiento Parabólico



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros del proyectil
5 v0, theta, g = 30, 45, 9.81
6 theta_rad = np.radians(theta)
7
8 # Tiempo de vuelo total
9 t_max = 2 * v0 * np.sin(theta_rad) / g
10 t = np.linspace(0, t_max, 100)
11
12 # Ecuaciones cinemáticas
13 x = v0 * np.cos(theta_rad) * t
14 y = v0 * np.sin(theta_rad) * t - 0.5 * g * t**2
15
16 # Crear subplots
17 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,
    ↪ 5))
18
19 # Altura vs tiempo
20 ax1.plot(t, y, 'b-', linewidth=2)
21 ax1.set_xlabel('Tiempo (s)')
22 ax1.set_ylabel('Altura (m)')
23 ax1.set_title('Altura vs Tiempo')
24 ax1.grid(True, alpha=0.3)
25
26 # Trayectoria 2D
27 ax2.plot(x, y, 'r-', linewidth=2)
28 ax2.set_xlabel('Distancia (m)')
29 ax2.set_ylabel('Altura (m)')
30 ax2.set_title('Trayectoria del Proyectil')
31 ax2.grid(True, alpha=0.3)
32
33 plt.tight_layout()
34 plt.show()
35
36 print(f"Alcance máximo: {np.max(x):.2f} m")
37 print(f"Altura máxima: {np.max(y):.2f} m")
```

Discusión: Las ecuaciones cinemáticas vectorizadas permiten calcular toda la trayectoria eficientemente.

Ejercicios Prácticos Integrados $\ni$ Solución 5 de Referencia:

Manipulación de Matrices

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Crear matriz 4x4 con números 1-16
5 matriz = np.arange(1, 17).reshape(4, 4)
6 diagonal = np.diag(matriz)
7 suma_diag = np.sum(diagonal)
8
9 print(f"Matriz 4x4:\n{matriz}")
10 print(f"Diagonal principal: {diagonal}")
11 print(f"Suma de la diagonal: {suma_diag}")
12
13 # Crear visualizaciones
14 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10,
    ↪ 4))
15
16 # Visualizar matriz como mapa de calor
17 im = ax1.imshow(matriz, cmap='viridis',
    ↪ interpolation='nearest')
```

```
18 ax1.set_title('Matriz 4x4')
19 for i in range(4):
20     for j in range(4):
21         ax1.text(j, i, matriz[i, j], ha='center',
22                 va='center', color='white',
23                 ↪ fontweight='bold')
24
25 plt.colorbar(im, ax=ax1)
26
27 # Gráfico de barras de la diagonal
28 ax2.bar(range(len(diagonal)), diagonal,
29         color='orange', alpha=0.7)
30 ax2.set_title('Diagonal Principal')
31 ax2.set_xlabel('Posición')
32 ax2.set_ylabel('Valor')
33 ax2.grid(True, alpha=0.3)
34
35 plt.tight_layout()
36 plt.show()
```

Discusión: La función `imshow` es ideal para visualizar matrices numéricas como mapas de calor.

Ejercicios Prácticos Integrados $\ni$ Solución 6 de Referencia:

Comparación de Distribuciones



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Configurar semilla para reproducibilidad
5 np.random.seed(42)
6
7 # Generar tres conjuntos de datos
8 datos1 = np.random.normal(0, 1, 1000)    # Normal
9      ↪ estándar
10 datos2 = np.random.normal(2, 0.5, 1000)  # Normal
11      ↪ desplazada
12 datos3 = np.random.uniform(0, 4, 1000)   #
13      ↪ Uniforme
14
15 # Calcular medias
16 media1, media2, media3 = np.mean(datos1),
17      ↪ np.mean(datos2), np.mean(datos3)
18
19 # Crear histogramas superpuestos
20 plt.figure(figsize=(10, 6))
21 plt.hist(datos1, bins=30, alpha=0.6,
22      ↪ label='N(0,1)', color='blue', density=True)
```

```
18 plt.hist(datos2, bins=30, alpha=0.6,
19      ↪ label='N(2,0.5)', color='red', density=True)
20 plt.hist(datos3, bins=30, alpha=0.6,
21      ↪ label='U(0,4)', color='green', density=True)
22
23 # Líneas verticales para las medias
24 plt.axvline(media1, color='blue', linestyle='--',
25      ↪ linewidth=2, label=f'Media N(0,1):
26      ↪ {media1:.2f}')
27 plt.axvline(media2, color='red', linestyle='--',
28      ↪ linewidth=2, label=f'Media N(2,0.5):
29      ↪ {media2:.2f}')
30 plt.axvline(media3, color='green', linestyle='--',
31      ↪ linewidth=2, label=f'Media U(0,4):
32      ↪ {media3:.2f}')
33
34 plt.legend()
35 plt.title('Comparación de Distribuciones
36      ↪ Estadísticas')
37 plt.xlabel('Valor')
38 plt.ylabel('Densidad')
39 plt.grid(True, alpha=0.3)
40 plt.show()
```

Discusión: Los histogramas con transparencia permiten comparar visualmente múltiples distribuciones.

- ¿Cuál de los tres ejercicios resultó más desafiante?
- ¿Qué diferencias notaron entre trabajar con listas y con arrays NumPy?
- ¿Cómo les resultó la personalización de las gráficas?
- ¿Qué estrategias utilizaron para debugging cuando algo no funcionaba?

Puntos Clave Observados

- NumPy simplifica las operaciones matemáticas vectoriales
- Matplotlib ofrece gran flexibilidad para personalización
- Los subplots son ideales para comparaciones múltiples

Cierre y Próximos Pasos

- Se realizó un **ejercicio evaluado** integrando NumPy y Matplotlib.
- Reforzamos **conceptos de la semana 6** (álgebra lineal, manipulación de arreglos, gráficas).
- Discutimos **resolución y dificultades** en la práctica.
- Seguiremos ampliando la **visualización** y el **manejo de datos** en las próximas semanas.

- **Semana 8:** Empezaremos **Unidad IV** (o V, según syllabus), enfocándonos en:
 - Programación orientada a objetos (si corresponde).
 - O profundizar en manejo de datos (pandas, estadística básica).
 - Más ejemplos de visualización avanzada (scatter 3D, contour, animaciones).
- **Retroalimentación individual** de esta evaluación se publicará en CANVAS.

- NumPy Docs - Sección `numpy.linalg`.
- Matplotlib Gallery - Ejemplos de todo tipo de gráficas (2D, 3D).
- CANVAS - Sección de entregas y feedback.

xcelente trabajoyhasta la próxima

- Recuerden revisar **notas y comentarios** en CANVAS.
- Continúen practicando la combinación NumPy + Matplotlib.
- ¡Nos vemos en la **Semana 8** con nuevos temas!